WebGPU and the price of compiling WGSL

Introducing a WebGPU info and timing tool

6-Oct-2025

Welcome, in the previous two posts I went looking for the WGSL limits (both in the spec, and also out of it).

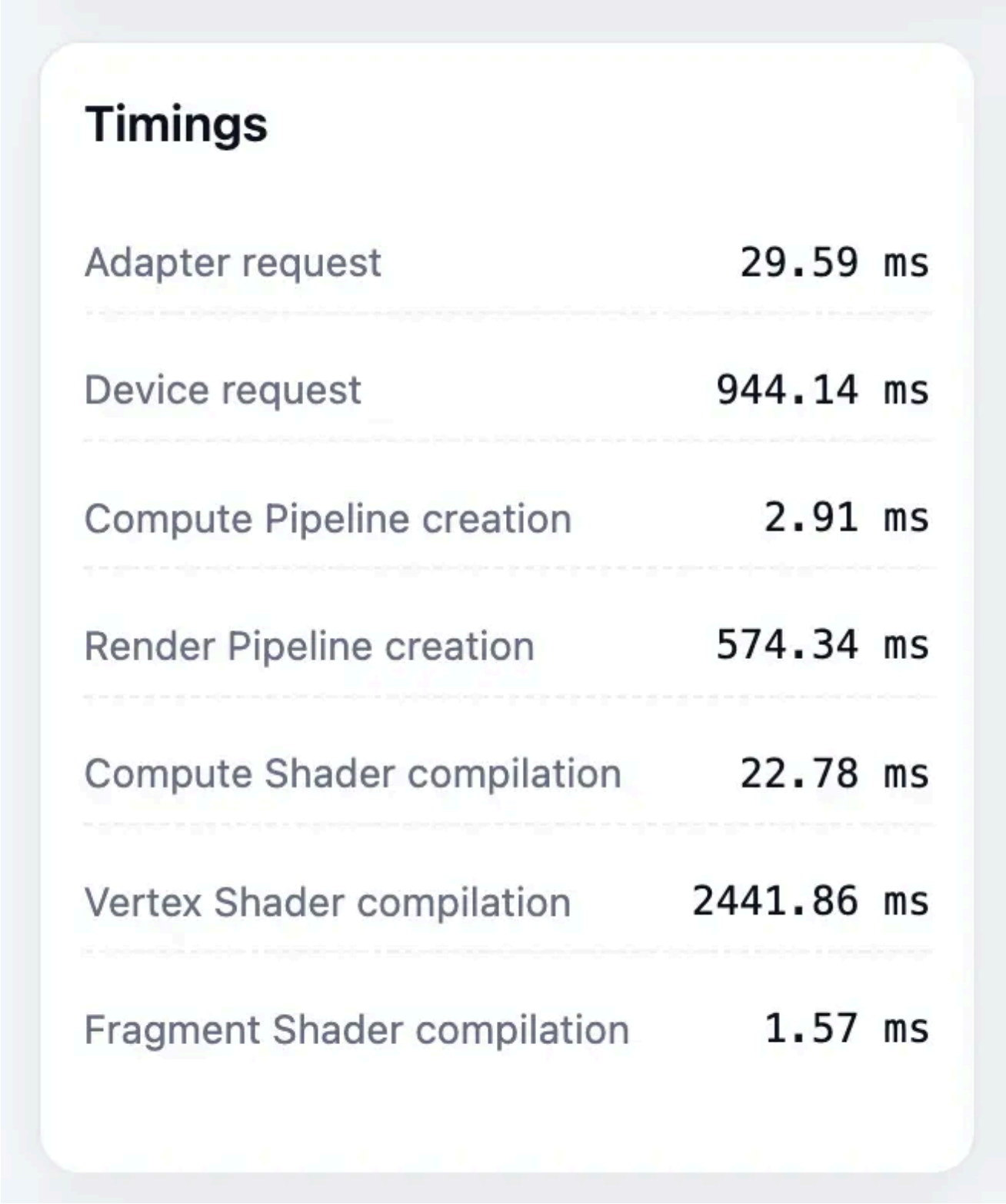
In this post I want to introduce a simple WebGPU app where you can create WGSL shaders by dragging some sliders to increase their complexity.

The idea is that compilation time and complexity can be known for the compute and rendering pipelines as well as in their different stages (vertex and fragment).

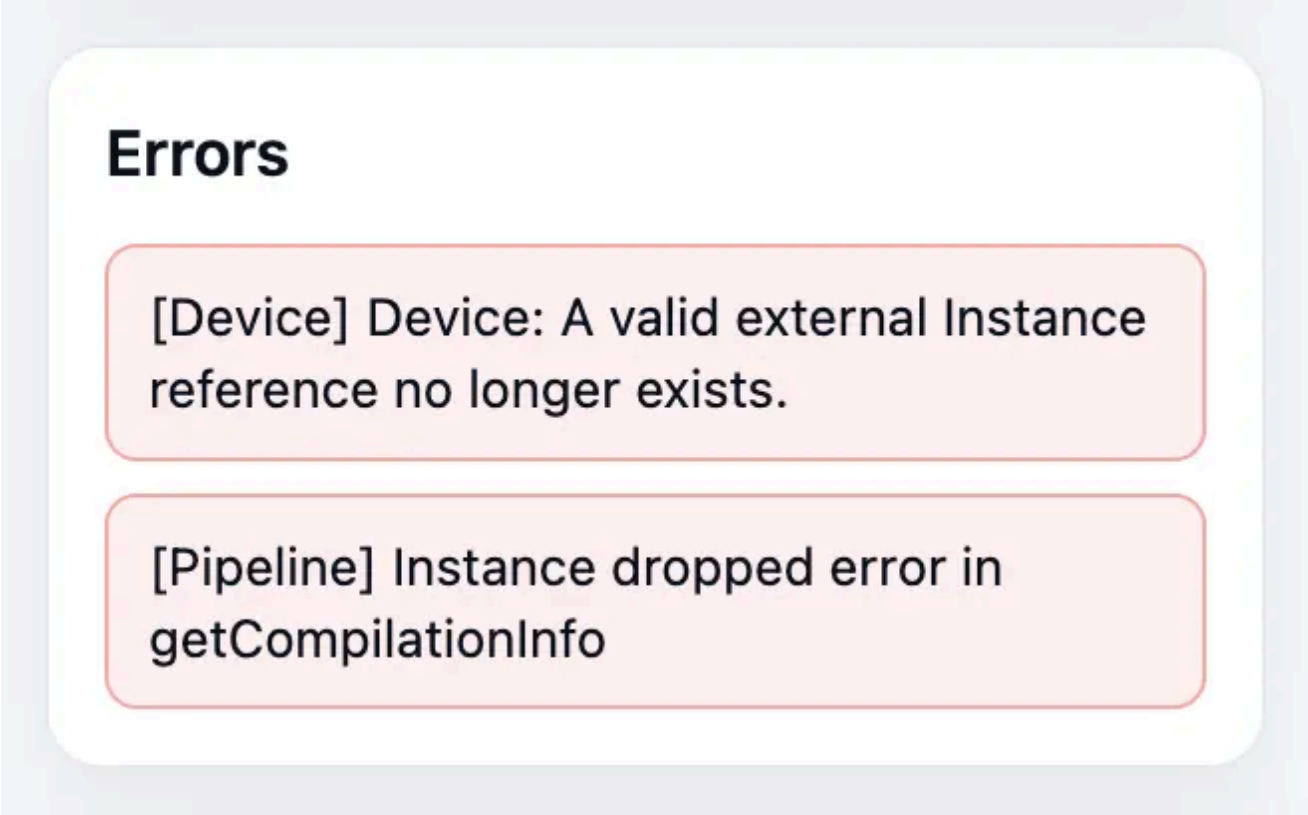
This is important to me because I wanted to know how much it costs to increase the shader complexity and where it might be worth to start considering using buffers/textures for some of it.

How long does it take to compile a 3Megabyte vertex shader?

Easy to know, here is the single result of the cold timings in Chrome (140.0.7339.215) from my Intel MacBook Pro (2019):



How about a 30Megabyte vertex shader??


Errors
[Device] Device: A valid external Instance reference no longer exists.
[Pipeline] Instance dropped error in getCompilationInfo

It errors. Ah!

WebGPU

The tool

A simple tool, along the lines of the awesome WebGPU report, but not as exhaustive.

You can find it here: WebGPU Diagnostics (and compilation benchmark) tool.

My intention was to compose it of 3 major parts:

I. A simple sample (a rotating triangle).
II. Shader complexity sliders and timings (compilation, adapter request and pipeline creation).
III. Device and adapter info (features, and announced limits)

Rotating triangle

The rotating triangle is made from the 3 initial shaders. A compute shader outputs vertices, then a vertex shader transforms them, and finally a fragment shader paints it.

0:00 / 0:04

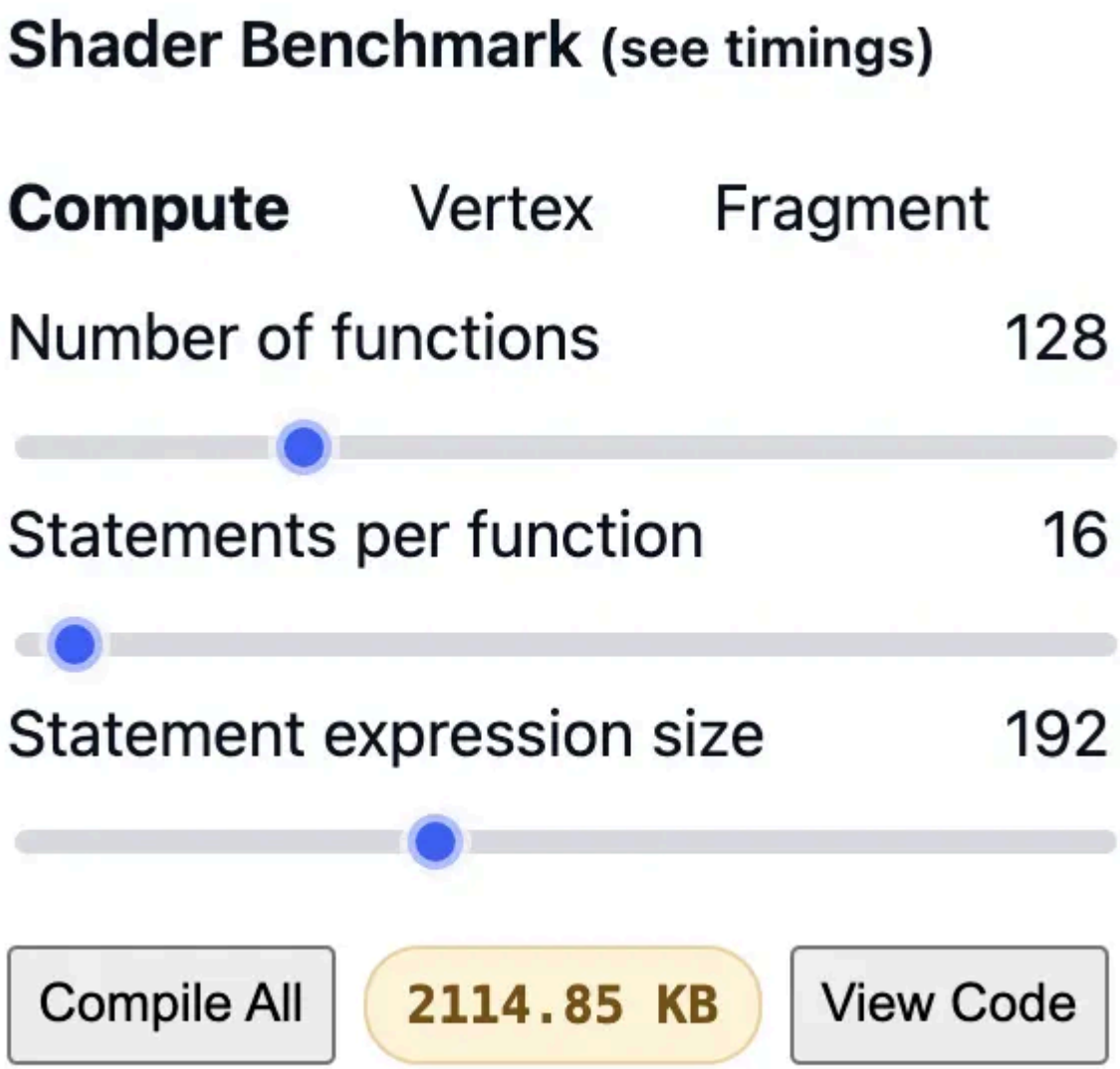
I think of this as a kind of "hello world" for WebGPU but also as a simple program that performs the happy path of the fastest rendering possible (or

close to it).

The idea along the app is that this triangle **never changes**. It always remains the same as the shader complexity increases.

Shader complexity sliders

Shader complexity can be increased at each pipeline stage. This includes separating the vertex shader from the fragment shader into different modules, to allow testing how much they differ (if they do at all).



The shader string is created before compilation starts. This is done in a WebWorker thread to keep the UI responsive (as much as possible) as memory and string traversal time increases.

I wanted to make these sliders biased towards complexity. A small change in them has to make the shader size increase by quite a bit. Making them purposefully unbalanced to ensure that device limits can be checked.

What WGSL code the sliders generate?

They are split into three categories:

I. Number of functions - dump n functions with 1 `f32` argument each that return the single variable declared in the body.
II. Number of statements - for each function in 1., creates n statements that just do a simple assignment.
III. Expression depth - for each statement in 2. expands it to have n sums.

So if we set them all to 8 (8 functions, each with 8 statements, each with 8 sums), it produces a 5Kilobyte file.

You can check the generator code here.

▶ Or click here to see the output produced with the three sliders at value 8

Device and adapter info

The features and limits is a nice thing to have in these kind of tools. They are not used, just communicated. A lot of them I don't know what they do, so I included a link to the corresponding WebGPU spec chapter.

An important part here is that I wanted to separate the *adapter* limits from the *device* limits. These can be different, and knowing these differences might be important when we are aiming for the upper end of the lot.

In WebGPU the adapter limits can be seen as the announced values of what might exist, while the device limits can be seen as what we have to work with.

These can be requested through the requiredLimits on creation.

Extra motivation

Another important part of motivation for doing this was to use boreDOM. A minimal JS framework that I created as a fun project that tries to do what major JS frameworks do but with a different approach just because why not? (no bundling, no compilation, no minification, and small size).

It was fun and cool to see that it works without bundling/compilation while handling WebGPU's complexity.

Conclusion

Go ahead, play with it a bit here and let me know how your device and browser reacts to massive shaders.

I found that there is a significant difference between cold and warm compilation times. This cold vs warm timing distinction could be interesting to explore further, and maybe it could even be leveraged/exploited to hide extremely complex shaders by splitting them in many small pipelines (just a wild hypothesis at this point i guess).

This post wraps the WGSL limits endeavour. For the next ones I might be introducing a rendering engine I am working that tries to perform with rendering the same spirit as boreDOM has to JS frameworks. Thanks for reading all of this and stay tuned!

GITHUB BSKY LINKEDIN E-MAIL